```
//// FILE: backend/app/main.py  (301 lines)

"""GeM BidVerify AI - HTTP API.

Run from the backend folder:   uvicorn app.main:app --reload --port 8000
Interactive docs:              http://localhost:8000/docs
All endpoints live under /api and (except /api/auth/* and /api/health) need a Bearer token.
"""
from typing import List, Literal, Optional

from fastapi import Depends, FastAPI, File, Form, Header, Query, Request, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, JSONResponse
from pydantic import BaseModel

from . import db, services
from .services import ServiceError

app = FastAPI(title="GeM BidVerify AI", version="2.0.0",
              description="AI-assisted bid compliance verification for GeM procurement officers.")
# The API uses bearer tokens (no cookies), so a permissive CORS policy is safe for local use.
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])


@app.on_event("startup")
def _startup():
    db.init_db()


@app.exception_handler(ServiceError)
async def _service_error(_: Request, exc: ServiceError):
    return JSONResponse(status_code=exc.status, content={"detail": exc.message})


def current_user(authorization: Optional[str] = Header(None)) -> dict:
    if not authorization or not authorization.lower().startswith("bearer "):
        raise ServiceError(401, "Authentication required.")
    return services.user_from_token(authorization.split(" ", 1)[1].strip())


# ---------------------------------------------------------------------------- request models
class LoginIn(BaseModel):
    username: str
    password: str


class OtpRequestIn(BaseModel):
    username: Optional[str] = None
    email: Optional[str] = None


class Verify2FAIn(BaseModel):
    username: Optional[str] = None
    email: Optional[str] = None
    otp: str


class ResetPasswordIn(BaseModel):
    username: Optional[str] = None
    email: Optional[str] = None
    otp: str
    new_password: str


class RegisterIn(BaseModel):
    username: str
    full_name: Optional[str] = None
    email: str
    password: str
    department: Optional[str] = None
    phone: Optional[str] = None


class RequirementIn(BaseModel):
    requirement_code: str
    requirement_name: str
    is_mandatory: bool = True
    verification_source: Optional[str] = None
    min_condition: Optional[str] = None
```

```
    weight: float = 10
    applicable_bidder_type: Optional[str] = "ALL"


class TenderIn(BaseModel):
    tender_id: Optional[str] = None
    title: str
    department: Optional[str] = ""
    tender_value: Optional[float] = None
    deadline: Optional[str] = None
    description: Optional[str] = ""
    requirements: List[RequirementIn] = []


class BidderIn(BaseModel):
    company_name: str
    cin: Optional[str] = None
    pan: Optional[str] = ""
    gstin: Optional[str] = None
    registered_address: Optional[str] = None
    contact_email: Optional[str] = None
    contact_phone: Optional[str] = None
    is_msme: bool = False
    msme_reg_no: Optional[str] = None
    is_startup: bool = False
    startup_dipp_no: Optional[str] = None


class BidIn(BaseModel):
    tender_id: int
    bidder_id: int


class ComplianceUpdateIn(BaseModel):
    points_awarded: float
    status: Literal["VERIFIED", "MISMATCH", "NON_COMPLIANT"]
    explanation: Optional[str] = None


class DecisionIn(BaseModel):
    decision: Literal["APPROVED", "REJECTED", "REQUIRES_CLARIFICATION"]
    comments: Optional[str] = ""
    clarification_details: Optional[str] = None


def _dump(model) -> dict:
    return model.model_dump() if hasattr(model, "model_dump") else model.dict()


# ---------------------------------------------------------------------------- health + auth
@app.get("/api/health")
def health():
    from . import extraction
    return {"status": "ok", "ocr_available": extraction.ocr_available()}


@app.post("/api/auth/login")
def login(body: LoginIn, request: Request):
    return services.login(body.username, body.password, request.client.host if request.client else "unknown")


@app.post("/api/auth/send-otp")
def send_otp(body: OtpRequestIn):
    return services.send_otp(body.username, body.email)


@app.post("/api/auth/verify-2fa")
def verify_2fa(body: Verify2FAIn):
    return services.verify_otp(body.username, body.email, body.otp)


@app.post("/api/auth/reset-password")
def reset_password(body: ResetPasswordIn):
    return services.reset_password(body.username, body.email, body.otp, body.new_password)


@app.post("/api/auth/register")
def register(body: RegisterIn):
    return services.register(_dump(body))
```

```
@app.get("/api/auth/me")
def me(user: dict = Depends(current_user)):
    return services._public_user(user)


# ---------------------------------------------------------------------------- dashboard
@app.get("/api/dashboard")
def dashboard(user: dict = Depends(current_user)):
    return services.dashboard()


# ---------------------------------------------------------------------------- tenders / bidders
@app.get("/api/tenders")
def list_tenders(user: dict = Depends(current_user)):
    return services.list_tenders()


@app.get("/api/tenders/{tender_ref}")
def get_tender(tender_ref: str, user: dict = Depends(current_user)):
    return services.get_tender(tender_ref)


@app.post("/api/tenders")
def create_tender(body: TenderIn, user: dict = Depends(current_user)):
    return services.create_tender(_dump(body), user)


@app.post("/api/tenders/{tender_id}/requirements")
def add_requirement(tender_id: int, body: RequirementIn, user: dict = Depends(current_user)):
    return services.add_requirement(tender_id, _dump(body), user)


@app.get("/api/bidders")
def list_bidders(user: dict = Depends(current_user)):
    return services.list_bidders()


@app.post("/api/bidders")
def create_bidder(body: BidderIn, user: dict = Depends(current_user)):
    return services.create_bidder(_dump(body), user)


# ---------------------------------------------------------------------------- bids
@app.get("/api/bids")
def list_bids(tender_id: Optional[int] = Query(None), user: dict = Depends(current_user)):
    return services.list_bids(tender_id)


@app.get("/api/bids/{bid_ref}")
def get_bid(bid_ref: str, user: dict = Depends(current_user)):
    return services.get_bid(bid_ref)


@app.post("/api/bids")
def create_bid(body: BidIn, user: dict = Depends(current_user)):
    return services.create_bid(body.tender_id, body.bidder_id, user)


@app.post("/api/bids/{bid_ref}/documents")
def upload_document(bid_ref: str, file: UploadFile = File(...), classified_type_hint: Optional[str] = Form(None)
    ,
                    user: dict = Depends(current_user)):
    return services.upload_document(bid_ref, file.filename, file.file.read(), classified_type_hint, user)


@app.delete("/api/bids/{bid_ref}/documents/{doc_id}")
def delete_document(bid_ref: str, doc_id: int, user: dict = Depends(current_user)):
    return services.delete_document(bid_ref, doc_id, user)


@app.get("/api/documents/{doc_id}/file")
def download_document(doc_id: int, user: dict = Depends(current_user)):
    path, name = services.document_file(doc_id)
    return FileResponse(path, filename=name)


@app.put("/api/bids/{bid_ref}/compliance/{result_id}")
```

```
def update_compliance(bid_ref: str, result_id: int, body: ComplianceUpdateIn, user: dict = Depends(current_user)
    ):
    return services.update_compliance_result(bid_ref, result_id, _dump(body), user)


@app.post("/api/bids/{bid_ref}/verify")
def verify_bid(bid_ref: str, user: dict = Depends(current_user)):
    return services.verify_bid(bid_ref, user)


@app.post("/api/bids/{bid_ref}/decision")
def decision(bid_ref: str, body: DecisionIn, user: dict = Depends(current_user)):
    return services.submit_decision(bid_ref, _dump(body), user)


# ---------------------------------------------------------------------------- integrations (simulated)
@app.get("/api/integrations/status")
def integrations_status(user: dict = Depends(current_user)):
    return services.integrations_status()


@app.post("/api/integrations/test-gst")
def test_gst(gstin: str, user: dict = Depends(current_user)):
    from . import adapters
    return adapters.lookup_gst(gstin)


@app.post("/api/integrations/test-pan")
def test_pan(pan: str, user: dict = Depends(current_user)):
    from . import adapters
    return adapters.lookup_pan(pan)


@app.post("/api/integrations/test-debarment")
def test_debarment(company_name: str, pan: str = "", user: dict = Depends(current_user)):
    from . import adapters
    return adapters.check_debarment(company_name, pan)


# ---------------------------------------------------------------------------- audit / notifications / admin
@app.get("/api/audit")
def audit(user: dict = Depends(current_user)):
    return services.audit_logs()


@app.get("/api/audit/verify")
def audit_verify(user: dict = Depends(current_user)):
    return services.audit_verify()


@app.get("/api/notifications")
def notifications(user: dict = Depends(current_user)):
    return services.notifications()


@app.post("/api/notifications/{notification_id}/read")
def read_notification(notification_id: int, user: dict = Depends(current_user)):
    return services.mark_notification_read(notification_id)


@app.get("/api/admin/export")
def export_data(user: dict = Depends(current_user)):
    return services.export_data()


@app.post("/api/admin/reset-demo")
def reset_demo(user: dict = Depends(current_user)):
    return services.reset_demo(user)


//// FILE: backend/app/config.py  (60 lines)

"""Runtime configuration.

Everything can be overridden with environment variables; the defaults are safe for a
local demo (SQLite file next to the code, OTP shown in the console).
"""
import os
import secrets
```

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = Path(os.getenv("GEM_DATA_DIR", BASE_DIR / "data"))
UPLOAD_DIR = DATA_DIR / "uploads"
DB_PATH = Path(os.getenv("GEM_DB_PATH", DATA_DIR / "gem_bidverify.db"))
for _p in (DATA_DIR, UPLOAD_DIR):
    _p.mkdir(parents=True, exist_ok=True)


def _load_secret() -> str:
    """JWT signing key: from GEM_SECRET_KEY, else generated once and kept in data/secret.key."""
    env = os.getenv("GEM_SECRET_KEY")
    if env:
        return env
    key_file = DATA_DIR / "secret.key"
    if not key_file.exists():
        key_file.write_text(secrets.token_hex(32))
    return key_file.read_text().strip()


SECRET_KEY = _load_secret()
TOKEN_TTL_HOURS = 8

OTP_TTL_SECONDS = 300
OTP_MAX_ATTEMPTS = 5
LOGIN_MAX_FAILURES = 5
LOGIN_LOCK_SECONDS = 300
MIN_PASSWORD_LENGTH = 6

MAX_UPLOAD_MB = 15
ALLOWED_EXTENSIONS = {".pdf", ".png", ".jpg", ".jpeg", ".tif", ".tiff", ".txt"}

# E-mail (optional). Without SMTP_HOST the OTP is printed in the server console.
SMTP_HOST = os.getenv("GEM_SMTP_HOST", "")
SMTP_PORT = int(os.getenv("GEM_SMTP_PORT", "587"))
SMTP_USER = os.getenv("GEM_SMTP_USER", "")
SMTP_PASSWORD = os.getenv("GEM_SMTP_PASSWORD", "")
SMTP_FROM = os.getenv("GEM_SMTP_FROM", SMTP_USER or "noreply@example.gov.in")
# For local demos the OTP is also returned to the browser when no mail server is set up.
SHOW_OTP_IN_RESPONSE = os.getenv("GEM_SHOW_OTP", "false" if SMTP_HOST else "true").lower() == "true"

DEMO_OFFICER_EMAIL = os.getenv("GEM_DEMO_OFFICER_EMAIL", "officer@example.gov.in")

# OCR
OCR_LANG = os.getenv("GEM_OCR_LANG", "eng")
TESSERACT_CMD = os.getenv("TESSERACT_CMD", "")
_WIN_TESSERACT = Path(r"C:\Program Files\Tesseract-OCR\tesseract.exe")
if not TESSERACT_CMD and _WIN_TESSERACT.exists():
    TESSERACT_CMD = str(_WIN_TESSERACT)

# If true a GSTIN whose check digit is wrong is treated as a failure (real GSTINs always pass).
STRICT_GSTIN_CHECKSUM = os.getenv("GEM_STRICT_GSTIN", "false").lower() == "true"


//// FILE: backend/app/db.py  (214 lines)

"""SQLite storage: schema, small helpers, hash-chained audit log, first-run demo data."""
import hashlib
import json
import sqlite3
from contextlib import contextmanager
from datetime import datetime, timezone
from pathlib import Path

from . import config, security

SCHEMA = """
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT UNIQUE NOT NULL, full_name TEXT NOT NULL,
  role TEXT NOT NULL, department TEXT, email TEXT, phone TEXT, password_hash TEXT NOT NULL,
  is_active INTEGER NOT NULL DEFAULT 1);
CREATE TABLE IF NOT EXISTS otps (
  key TEXT PRIMARY KEY, otp_hash TEXT NOT NULL, email TEXT, expires_at REAL NOT NULL, attempts INTEGER NOT NULL
    DEFAULT 0);
CREATE TABLE IF NOT EXISTS tenders (
  id INTEGER PRIMARY KEY AUTOINCREMENT, tender_id TEXT UNIQUE NOT NULL, title TEXT NOT NULL, department TEXT,
  tender_value REAL, deadline TEXT, status TEXT NOT NULL DEFAULT 'ACTIVE', description TEXT, created_at TEXT NOT
     NULL);
```

```
CREATE TABLE IF NOT EXISTS requirements (
  id INTEGER PRIMARY KEY AUTOINCREMENT, tender_pk INTEGER NOT NULL REFERENCES tenders(id) ON DELETE CASCADE,
  requirement_code TEXT NOT NULL, requirement_name TEXT NOT NULL, is_mandatory INTEGER NOT NULL DEFAULT 1,
  verification_source TEXT, min_condition TEXT, weight REAL NOT NULL DEFAULT 10, applicable_bidder_type TEXT DEF
    AULT 'ALL');
CREATE TABLE IF NOT EXISTS bidders (
  id INTEGER PRIMARY KEY AUTOINCREMENT, company_name TEXT NOT NULL, cin TEXT, pan TEXT, gstin TEXT,
  registered_address TEXT, contact_email TEXT, contact_phone TEXT, is_msme INTEGER DEFAULT 0, msme_reg_no TEXT,
  is_startup INTEGER DEFAULT 0, startup_dipp_no TEXT);
CREATE TABLE IF NOT EXISTS bids (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_id TEXT UNIQUE NOT NULL,
  tender_pk INTEGER NOT NULL REFERENCES tenders(id), bidder_pk INTEGER NOT NULL REFERENCES bidders(id),
  submission_date TEXT NOT NULL, status TEXT NOT NULL, compliance_score INTEGER DEFAULT 0, risk_level TEXT DEFAU
    LT 'LOW',
  ai_recommendation TEXT, ai_explanation TEXT, risk_score INTEGER DEFAULT 0, risk_factors TEXT DEFAULT '[]',
  recommended_action TEXT);
CREATE TABLE IF NOT EXISTS documents (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER NOT NULL REFERENCES bids(id) ON DELETE CASCADE,
  file_name TEXT NOT NULL, file_type TEXT, file_size_kb INTEGER, stored_path TEXT, classified_type TEXT,
  classification_confidence REAL, ocr_status TEXT, extracted_fields TEXT DEFAULT '[]', created_at TEXT NOT NULL)
    ;
CREATE TABLE IF NOT EXISTS compliance_results (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER NOT NULL REFERENCES bids(id) ON DELETE CASCADE,
  requirement_code TEXT NOT NULL, requirement_name TEXT, status TEXT NOT NULL, points_awarded REAL, max_points R
    EAL,
  evidence_document TEXT, confidence_score REAL, rule_applied TEXT, explanation TEXT, flags TEXT DEFAULT '[]',
  is_mandatory INTEGER DEFAULT 1, officer_override INTEGER DEFAULT 0);
CREATE TABLE IF NOT EXISTS decisions (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER UNIQUE NOT NULL REFERENCES bids(id) ON DELETE CASCADE,
  officer_name TEXT, decision TEXT NOT NULL, comments TEXT, clarification_details TEXT, decided_at TEXT NOT NULL
    );
CREATE TABLE IF NOT EXISTS audit_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_id INTEGER, user_name TEXT, action TEXT NOT NULL, entity_type TEXT,
  details TEXT, rule_applied TEXT, ip_address TEXT, timestamp TEXT NOT NULL, prev_hash TEXT, entry_hash TEXT);
CREATE TABLE IF NOT EXISTS notifications (
  id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT, message TEXT, category TEXT, is_read INTEGER DEFAULT 0,
  link TEXT, created_at TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS connectors (
  id TEXT PRIMARY KEY, name TEXT, type TEXT, mode TEXT, status TEXT, latency_ms INTEGER, badge TEXT);
CREATE TABLE IF NOT EXISTS evaluations (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER NOT NULL, evaluated_at TEXT NOT NULL, score INTEGER,
  risk_level TEXT, recommendation TEXT);
"""


def now_iso() -> str:
    return datetime.now(timezone.utc).isoformat(timespec="milliseconds").replace("+00:00", "Z")


@contextmanager
def connect():
    conn = sqlite3.connect(config.DB_PATH, timeout=30)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys=ON")
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()


def rows(conn, sql, params=()):
    return [dict(r) for r in conn.execute(sql, params).fetchall()]


def row(conn, sql, params=()):
    r = conn.execute(sql, params).fetchone()
    return dict(r) if r else None


def loads(value, default):
    try:
        return json.loads(value) if value else default
    except (TypeError, ValueError):
        return default
```

```
# ---------------------------------------------------------------------------------------
# Tamper-evident audit log: every entry stores SHA-256(previous hash + entry content).
# Changing or deleting an old row breaks the chain, which verify_audit_chain() detects.
# ---------------------------------------------------------------------------------------
def _entry_hash(prev_hash: str, entry: dict) -> str:
    payload = json.dumps(entry, sort_keys=True, ensure_ascii=False)
    return hashlib.sha256((prev_hash + payload).encode()).hexdigest()


def add_audit(conn, action, details, entity_type="BID", bid_id=None, user_name="System",
              rule_applied=None, ip_address="127.0.0.1", timestamp=None):
    last = row(conn, "SELECT entry_hash FROM audit_logs ORDER BY id DESC LIMIT 1")
    prev = last["entry_hash"] if last and last["entry_hash"] else "GENESIS"
    entry = {"bid_id": bid_id, "user_name": user_name, "action": action, "entity_type": entity_type,
             "details": details, "rule_applied": rule_applied, "ip_address": ip_address,
             "timestamp": timestamp or now_iso()}
    conn.execute(
        "INSERT INTO audit_logs (bid_id,user_name,action,entity_type,details,rule_applied,ip_address,timestamp,p
    rev_hash,entry_hash)"
        " VALUES (:bid_id,:user_name,:action,:entity_type,:details,:rule_applied,:ip_address,:timestamp,:prev,:h
    )",
        {**entry, "prev": prev, "h": _entry_hash(prev, entry)})


def verify_audit_chain(conn) -> dict:
    prev, count = "GENESIS", 0
    for r in conn.execute("SELECT * FROM audit_logs ORDER BY id"):
        r = dict(r)
        entry = {k: r[k] for k in ("bid_id", "user_name", "action", "entity_type", "details",
                                   "rule_applied", "ip_address", "timestamp")}
        if r["prev_hash"] != prev or r["entry_hash"] != _entry_hash(prev, entry):
            return {"valid": False, "entries_checked": count, "first_broken_id": r["id"]}
        prev, count = r["entry_hash"], count + 1
    return {"valid": True, "entries_checked": count, "first_broken_id": None}


# ---------------------------------------------------------------------------------------
# Schema + demo data
# ---------------------------------------------------------------------------------------
def init_db():
    with connect() as conn:
        conn.executescript(SCHEMA)
        if not row(conn, "SELECT id FROM users LIMIT 1"):
            seed_demo_data(conn)


def _b(v):
    return 1 if v else 0


def seed_demo_data(conn):
    """Load the demo tenders, bidders, bids and users that ship with the project."""
    seeds = json.loads((Path(__file__).parent / "seed_data" / "seeds.json").read_text(encoding="utf-8"))

    for u in seeds["seedUsers"]:
        conn.execute(
            "INSERT INTO users (id,username,full_name,role,department,email,phone,password_hash,is_active) VALUE
    S (?,?,?,?,?,?,?,?,1)",
            (u["id"], u["username"], u["full_name"], u["role"], u["department"], config.DEMO_OFFICER_EMAIL,
             u.get("phone"), security.hash_password(u["password"])))

    for t in seeds["seedTenders"]:
        conn.execute(
            "INSERT INTO tenders (id,tender_id,title,department,tender_value,deadline,status,description,created
    _at) VALUES (?,?,?,?,?,?,?,?,?)",
            (t["id"], t["tender_id"], t["title"], t["department"], t["tender_value"], t["deadline"], t["status"]
    ,
             t.get("description"), t["created_at"]))
        for r in t["requirements"]:
            conn.execute(
                "INSERT INTO requirements (tender_pk,requirement_code,requirement_name,is_mandatory,verification
    _source,min_condition,weight,applicable_bidder_type) VALUES (?,?,?,?,?,?,?,?)",
                (t["id"], r["requirement_code"], r["requirement_name"], _b(r["is_mandatory"]), r["verification_s
    ource"],
                 r["min_condition"], r["weight"], r.get("applicable_bidder_type", "ALL")))

    for b in seeds["seedBidders"]:
        conn.execute(
```

```
            "INSERT INTO bidders (id,company_name,cin,pan,gstin,registered_address,contact_email,contact_phone,i
    s_msme,msme_reg_no,is_startup,startup_dipp_no) VALUES (?,?,?,?,?,?,?,?,?,?,?,?)",
            (b["id"], b["company_name"], b.get("cin"), b.get("pan"), b.get("gstin"), b.get("registered_address")
    ,
             b.get("contact_email"), b.get("contact_phone"), _b(b.get("is_msme")), b.get("msme_reg_no"),
             _b(b.get("is_startup")), b.get("startup_dipp_no")))

    for bid in seeds["seedBids"]:
        ra = bid["risk_assessment"]
        conn.execute(
            "INSERT INTO bids (id,bid_id,tender_pk,bidder_pk,submission_date,status,compliance_score,risk_level,
    ai_recommendation,ai_explanation,risk_score,risk_factors,recommended_action) VALUES (?,?,?,?,?,?,?,?,?,?,?,?
    ,?)",
            (bid["id"], bid["bid_id"], bid["tender_id"], bid["bidder_id"], bid["submission_date"], bid["status"]
    ,
             bid["compliance_score"], bid["risk_level"], bid["ai_recommendation"], bid["ai_explanation"],
             ra["risk_score"], json.dumps(ra["risk_factors"]), ra["recommended_action"]))
        for d in bid["documents"]:
            conn.execute(
                "INSERT INTO documents (id,bid_pk,file_name,file_type,file_size_kb,stored_path,classified_type,c
    lassification_confidence,ocr_status,extracted_fields,created_at) VALUES (?,?,?,?,?,?,?,?,?,?,?)",
                (d["id"], bid["id"], d["file_name"], d["file_type"], d["file_size_kb"], None, d["classified_type
    "],
                 d["classification_confidence"], d["ocr_status"], json.dumps(d["extracted_fields"]), d["created_
    at"]))
        for c in bid["compliance_results"]:
            conn.execute(
                "INSERT INTO compliance_results (id,bid_pk,requirement_code,requirement_name,status,points_award
    ed,max_points,evidence_document,confidence_score,rule_applied,explanation,flags) VALUES (?,?,?,?,?,?,?,?,?,?
    ,?,'[]')",
                (c["id"], bid["id"], c["requirement_code"], c["requirement_name"], c["status"], c["points_awarde
    d"],
                 c["max_points"], c["evidence_document"], c["confidence_score"], c["rule_applied"], c["explanati
    on"]))
        conn.execute("INSERT INTO evaluations (bid_pk,evaluated_at,score,risk_level,recommendation) VALUES (?,?,
    ?,?,?)",
                     (bid["id"], bid["submission_date"], bid["compliance_score"], bid["risk_level"], bid["ai_rec
    ommendation"]))

    for a in seeds["seedAuditLogs"]:
        add_audit(conn, a["action"], a["details"], a.get("entity_type", "BID"), None, a["user_name"],
                  a.get("rule_applied"), a.get("ip_address", "127.0.0.1"), a["timestamp"])
    for n in seeds["seedNotifications"]:
        conn.execute("INSERT INTO notifications (id,title,message,category,is_read,link,created_at) VALUES (?,?,
    ?,?,?,?,?)",
                     (n["id"], n["title"], n["message"], n["category"], _b(n["is_read"]), n.get("link"), n["crea
    ted_at"]))
    for c in seeds["seedConnectors"]:
        conn.execute("INSERT INTO connectors (id,name,type,mode,status,latency_ms,badge) VALUES (?,?,?,?,?,?,?)"
    ,
                     (c["id"], c["name"], c["type"], c["mode"], c["status"], c["latency_ms"], "Simulated"))


def reset_demo_data():
    """Drop everything and reload the demo data (uploaded files are removed as well)."""
    with connect() as conn:
        for table in ("evaluations", "decisions", "compliance_results", "documents", "bids", "requirements",
                      "tenders", "bidders", "audit_logs", "notifications", "connectors", "otps", "users"):
            conn.execute(f"DELETE FROM {table}")
        conn.execute("DELETE FROM sqlite_sequence")
        seed_demo_data(conn)
        add_audit(conn, "DATA_RESET_DEFAULT", "Demo data restored to the shipped baseline.", "SYSTEM", None, "Sy
    stem")
    for f in config.UPLOAD_DIR.glob("**/*"):
        if f.is_file():
            f.unlink()


//// FILE: backend/app/security.py  (64 lines)

"""Password hashing, JWT tokens and one-time passwords (standard library + PyJWT)."""
import base64
import hashlib
import hmac
import secrets
import time

import jwt
```

```
from . import config

_ITERATIONS = 200_000


def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, _ITERATIONS)
    return "pbkdf2_sha256${}${}${}".format(
        _ITERATIONS, base64.b64encode(salt).decode(), base64.b64encode(digest).decode()
    )


def verify_password(password: str, stored: str) -> bool:
    try:
        _, iterations, salt_b64, digest_b64 = stored.split("$")
        salt = base64.b64decode(salt_b64)
        expected = base64.b64decode(digest_b64)
        actual = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, int(iterations))
        return hmac.compare_digest(actual, expected)
    except (ValueError, TypeError):
        return False


def create_token(user: dict) -> str:
    now = int(time.time())
    payload = {
        "sub": str(user["id"]),
        "username": user["username"],
        "iat": now,
        "exp": now + config.TOKEN_TTL_HOURS * 3600,
    }
    return jwt.encode(payload, config.SECRET_KEY, algorithm="HS256")


def decode_token(token: str) -> dict:
    """Return the token payload or raise jwt.InvalidTokenError."""
    return jwt.decode(token, config.SECRET_KEY, algorithms=["HS256"])


def generate_otp() -> str:
    return str(100000 + secrets.randbelow(900000))


def hash_otp(otp: str) -> str:
    return hmac.new(config.SECRET_KEY.encode(), otp.encode(), hashlib.sha256).hexdigest()


def mask_email(email: str) -> str:
    if "@" not in email:
        return email
    local, domain = email.split("@", 1)
    if len(local) <= 2:
        return f"{local}*@{domain}"
    return local[:2] + "*" * max(2, len(local) - 3) + local[-1] + "@" + domain


//// FILE: backend/app/services.py  (644 lines)

"""Business logic. Every function returns plain dictionaries shaped exactly like the frontend expects."""
import json
import random
import re
import smtplib
import time
import uuid
from datetime import datetime, timedelta, timezone
from email.message import EmailMessage
from pathlib import Path

from . import adapters, config, db, extraction, rules, security
from .db import connect, loads, now_iso, row, rows
from .extraction import PAN_RE, GSTIN_RE


class ServiceError(Exception):
    def __init__(self, status: int, message: str):
        super().__init__(message)
        self.status, self.message = status, message
```

```
def _num(x):
    """JSON-friendly number: 10.0 -> 10, 7.5 -> 7.5"""
    if x is None:
        return None
    return int(x) if float(x).is_integer() else round(float(x), 2)


# ============================================================================================
# Authentication
# ============================================================================================
_login_failures = {}  # username -> (count, locked_until)


def _public_user(u: dict) -> dict:
    return {"id": u["id"], "username": u["username"], "full_name": u["full_name"], "role": u["role"],
            "department": u["department"], "email": u["email"], "phone": u["phone"], "is_active": bool(u["is_act
    ive"])}


def _session(u: dict) -> dict:
    return {"access_token": security.create_token(u), "token_type": "bearer", "user": _public_user(u)}


def _find_user(conn, username=None, email=None):
    if username:
        u = row(conn, "SELECT * FROM users WHERE username=? OR lower(email)=lower(?)", (username, username))
        if u:
            return u
    if email:
        return row(conn, "SELECT * FROM users WHERE lower(email)=lower(?)", (email,))
    return None


def user_from_token(token: str) -> dict:
    import jwt
    try:
        payload = security.decode_token(token)
    except jwt.InvalidTokenError:
        raise ServiceError(401, "Session expired or invalid. Please sign in again.")
    with connect() as conn:
        u = row(conn, "SELECT * FROM users WHERE id=? AND is_active=1", (int(payload["sub"]),))
    if not u:
        raise ServiceError(401, "Account not found or disabled.")
    return u


def login(username: str, password: str, ip: str = "unknown") -> dict:
    count, locked_until = _login_failures.get(username, (0, 0))
    if locked_until > time.time():
        raise ServiceError(429, "Too many failed attempts. Try again in a few minutes.")
    with connect() as conn:
        u = row(conn, "SELECT * FROM users WHERE username=? AND is_active=1", (username,))
        if not u or not security.verify_password(password, u["password_hash"]):
            count += 1
            lock = time.time() + config.LOGIN_LOCK_SECONDS if count >= config.LOGIN_MAX_FAILURES else 0
            _login_failures[username] = (0 if lock else count, lock)
            db.add_audit(conn, "LOGIN_FAILED", f"Failed sign-in attempt for '{username}'.", "AUTH", None, userna
    me, ip_address=ip)
            conn.commit()
            raise ServiceError(401, "Invalid username or password.")
        _login_failures.pop(username, None)
        db.add_audit(conn, "OFFICER_LOGIN", f"Officer '{u['full_name']}' authenticated via GeM console.", "AUTH"
    ,
                     None, u["full_name"], ip_address=ip)
        return _session(u)


def _send_mail(to: str, subject: str, body: str):
    msg = EmailMessage()
    msg["From"], msg["To"], msg["Subject"] = config.SMTP_FROM, to, subject
    msg.set_content(body)
    with smtplib.SMTP(config.SMTP_HOST, config.SMTP_PORT, timeout=20) as smtp:
        smtp.starttls()
        if config.SMTP_USER:
            smtp.login(config.SMTP_USER, config.SMTP_PASSWORD)
        smtp.send_message(msg)
```

```
    code: 'MAKE_IN_INDIA',
    name: 'Make in India (MII) Local Content Declaration',
    mandatory: true,
    source: 'DPIIT_MII_DECLARATION',
    min_cond: 'Class-I Local Supplier (>=50% local content)',
    weight: 15,
  },
  {
    code: 'EPFO',
    name: 'EPFO Registration & ECR Electronic Challan',
    mandatory: false,
    source: 'EPFO_PORTAL',
    min_cond: 'Active establishment code',
    weight: 5,
  },
  {
    code: 'ESIC',
    name: 'ESIC Registration & Compliance Declaration',
    mandatory: false,
    source: 'ESIC_PORTAL',
    min_cond: 'Active ESIC employer registration',
    weight: 5,
  },
  {
    code: 'DEBARMENT',
    name: 'Non-Blacklisting / Debarment Affidavit',
    mandatory: true,
    source: 'CPPP_GEM_DEBARMENT_LIST',
    min_cond: 'No debarment by any Central/State Govt/PSU',
    weight: 10,
  },
  {
    code: 'BIS',
    name: 'BIS Quality Standard / CRS Certification',
    mandatory: false,
    source: 'BIS_PORTAL',
    min_cond: 'Valid IS/ISO Standard certification',
    weight: 10,
  },
  {
    code: 'STARTUP',
    name: 'DPIIT Startup Recognition Certificate',
    mandatory: false,
    source: 'STARTUP_INDIA_PORTAL',
    min_cond: 'Valid DIPP recognition for EMD exemption',
    weight: 5,
  },
];

const RequirementsEditor = ({ requirements: e, onChange: t }) => {
  const [r, n] = useState(''),
    [i, a] = useState(''),
    [o, l] = useState(true),
    [c, u] = useState(10),
    [f, d] = useState(''),
    p = (g) => {
      const v = e.some((m) => m.requirement_code === g.code);
      t(
        v
          ? e.filter((m) => m.requirement_code !== g.code)
          : [
              ...e,
              {
                requirement_code: g.code,
                requirement_name: g.name,
                is_mandatory: g.mandatory,
                verification_source: g.source,
                min_condition: g.min_cond,
                weight: g.weight,
                applicable_bidder_type: 'ALL',
              },
            ],
      );
    },
    h = (g) => {
      (g.preventDefault(),
        !(!r || !i) &&
          (t([
            ...e,
```

```
          {
            requirement_code: r.toUpperCase().replace(/\s+/g, '_'),
            requirement_name: i,
            is_mandatory: o,
            verification_source: 'PORTAL_CROSS_CHECK',
            min_condition: f || undefined,
            weight: c,
            applicable_bidder_type: 'ALL',
          },
        ]),
        n(''),
        a(''),
        d('')));
  },
  y = (g) => {
    t(e.filter((v) => v.requirement_code !== g));
  },
  x = e.reduce((g, v) => g + Number(v.weight), 0);
return (
  <div className="space-y-5">
    <div>
      <div className="flex justify-between items-center mb-3">
        <label className="text-xs font-bold uppercase tracking-wider text-slate-700 flex items-center gap-1.5"
  >
          <SlidersVerticalIcon className="w-3.5 h-3.5 text-pine-700" />
          Standard Statutory Criteria (Click to Toggle)
        </label>
        <span
          className={`text-xs font-mono font-bold px-3 py-1 rounded-full ${x === 100 ? 'bg-pine-100 text-pine-
  800' : 'bg-champagne-100 text-champagne-800'}`}
        >
          {'Total Scoring Weight: '}
          {x}
          {' pts'}
        </span>
      </div>
      <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 gap-2.5">
        {defaultRequirementTemplates.map((g) => {
          const v = e.some((m) => m.requirement_code === g.code);
          return (
            <button
              key={g.code}
              type="button"
              onClick={() => p(g)}
              className={`p-3 text-left rounded-2xl border text-xs transition-all flex flex-col justify-betwee
  n ${v ? 'bg-pine-50/90 border-pine-600 text-pine-950 shadow-sm font-semibold ring-1 ring-pine-600' : 'bg-whi
  te border-slate-200 text-slate-700 hover:bg-slate-50'}`}
            >
              <div className="flex justify-between items-start mb-1">
                <span className="font-mono text-[11px] font-bold text-champagne-800">
                  {g.code}
                </span>
                <span className="text-[10px] bg-slate-100 text-slate-700 px-1.5 py-0.5 rounded font-mono font-
  bold">
                  {g.weight}
                  {' pts'}
                </span>
              </div>
              <p className="text-[11px] leading-tight line-clamp-2 mt-0.5">{g.name}</p>
              <div className="mt-2.5 pt-1.5 border-t border-slate-100 flex justify-between text-[9.5px] text-s
  late-400 font-medium">
                <span>{g.mandatory ? 'Mandatory' : 'Optional'}</span>
                <span className={v ? 'text-pine-700 font-bold' : ''}>
                  {v ? '  Enabled' : '+ Add'}
                </span>
              </div>
            </button>
          );
        })}
      </div>
    </div>
    <div className="bg-white rounded-2xl border border-slate-200 overflow-hidden shadow-card-soft">
      <div className="px-4 py-3 bg-slate-50 border-b border-slate-200 flex justify-between items-center">
        <span className="text-xs font-bold text-slate-800">
          Active Tender Evaluation Criteria ({e.length})
        </span>
      </div>
      <div className="divide-y divide-slate-100 max-h-56 overflow-y-auto custom-scrollbar">
        {e.length === 0 ? (
```

```
        <div className="p-5 text-center text-xs text-slate-500">
          No criteria selected. Choose from presets above or add a custom requirement below.
        </div>
      ) : (
        e.map((g) => (
          <div
            key={g.requirement_code}
            className="p-3.5 flex items-center justify-between text-xs hover:bg-slate-50 transition-colors"
          >
            <div className="flex items-center gap-3 min-w-0">
              <span className="font-mono text-xs font-bold bg-slate-100 text-slate-900 px-2.5 py-0.5 rounded
-lg border border-slate-200 flex-shrink-0">
                {g.requirement_code}
              </span>
              <div className="min-w-0">
                <p className="font-bold text-slate-900 truncate">{g.requirement_name}</p>
                <p className="text-[11px] text-slate-500 truncate">
                  {g.min_condition || 'Standard verification'}
                </p>
              </div>
            </div>
            <div className="flex items-center gap-3.5 flex-shrink-0">
              <span
                className={`text-[10px] font-bold px-2 py-0.5 rounded-full ${g.is_mandatory ? 'bg-rose-100 t
ext-rose-800' : 'bg-slate-100 text-slate-700'}`}
              >
                {g.is_mandatory ? 'Mandatory' : 'Optional'}
              </span>
              <span className="font-mono font-bold text-slate-800 bg-slate-50 px-2 py-0.5 rounded border bor
der-slate-200">
                {g.weight}
                {' pts'}
              </span>
              <button
                type="button"
                onClick={() => y(g.requirement_code)}
                className="text-slate-400 hover:text-rose-600 p-1 rounded-lg transition-colors"
                title="Remove criterion"
              >
                <Trash2Icon className="w-4 h-4" />
              </button>
            </div>
          </div>
        ))
      )}
    </div>
  </div>
  <div className="p-4 bg-slate-50 rounded-2xl border border-slate-200 space-y-3">
    <h5 className="text-xs font-bold text-slate-800 flex items-center gap-1.5">
      <Settings2Icon className="w-4 h-4 text-pine-700" />
      Add Custom Tender-Specific Requirement
    </h5>
    <div className="grid grid-cols-1 sm:grid-cols-4 gap-3 text-xs">
      <input
        type="text"
        placeholder="Code (e.g. ISO_27001)"
        value={r}
        onChange={(g) => n(g.target.value)}
        className="p-2.5 border border-slate-300 rounded-xl bg-white focus:ring-2 focus:ring-teal-600 focus:
outline-none"
      />
      <input
        type="text"
        placeholder="Requirement Name"
        value={i}
        onChange={(g) => a(g.target.value)}
        className="p-2.5 border border-slate-300 rounded-xl bg-white sm:col-span-2 focus:ring-2 focus:ring-t
eal-600 focus:outline-none"
      />
      <input
        type="number"
        placeholder="Weight (pts)"
        value={c}
        onChange={(g) => u(Number(g.target.value))}
        className="p-2.5 border border-slate-300 rounded-xl bg-white font-mono focus:ring-2 focus:ring-teal-
600 focus:outline-none"
      />
    </div>
    <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-2 pt-1">
```

```
          <label className="flex items-center gap-2 text-xs text-slate-700 cursor-pointer select-none">
            <input
              type="checkbox"
              checked={o}
              onChange={(g) => l(g.target.checked)}
              className="rounded text-teal-600 focus:ring-teal-500"
            />
            <span className="font-medium">
              Mandatory Requirement (Non-compliance disqualifies bidder)
            </span>
          </label>
          <button
            type="button"
            onClick={h}
            className="px-4 py-2 bg-pine-800 hover:bg-pine-900 text-white rounded-xl font-bold text-xs flex item
    s-center gap-1.5 shadow-sm transition-all"
          >
            <PlusIcon className="w-3.5 h-3.5 text-champagne-300" />
            <span>Add Criterion</span>
          </button>
        </div>
      </div>
    </div>
  );
};

export const TendersPage = () => {
  const [e, t] = useState([]),
    [r, n] = useState(true),
    [i, a] = useState(false),
    [o, l] = useState(''),
    [c, u] = useState('GEM/2026/B/' + Math.floor(1e4 + Math.random() * 9e4)),
    [f, d] = useState(''),
    [p, h] = useState(''),
    [y, x] = useState(5e6),
    [g, v] = useState(new Date(Date.now() + 14 * 864e5).toISOString().split('T')[0]),
    [m, b] = useState(''),
    [j, w] = useState([
      {
        requirement_code: 'PAN',
        requirement_name: 'Permanent Account Number (PAN) Card',
        is_mandatory: true,
        verification_source: 'ITD_PAN_REGISTRY',
        min_condition: 'Valid active PAN in bidder name',
        weight: 10,
        applicable_bidder_type: 'ALL',
      },
      {
        requirement_code: 'GST',
        requirement_name: 'Goods & Services Tax (GST) Registration',
        is_mandatory: true,
        verification_source: 'GSTN_PORTAL',
        min_condition: 'Active GSTIN with regular filing',
        weight: 15,
        applicable_bidder_type: 'ALL',
      },
      {
        requirement_code: 'UDYAM',
        requirement_name: 'MSME / Udyam Registration Certificate',
        is_mandatory: false,
        verification_source: 'UDYAM_PORTAL',
        min_condition: 'Valid Udyam reg for purchase preference',
        weight: 10,
        applicable_bidder_type: 'ALL',
      },
      {
        requirement_code: 'ITR',
        requirement_name: 'Income Tax Returns & CA Turnover (Last 3 FYs)',
        is_mandatory: true,
        verification_source: 'ITD_ITR_ICAI_UDIN',
        min_condition: 'Min average turnover ₹1.5 Cr with valid UDIN',
        weight: 15,
        applicable_bidder_type: 'ALL',
      },
      {
        requirement_code: 'OEM_AUTH',
        requirement_name: 'Manufacturer Authorization Form (MAF / OEM Auth)',
        is_mandatory: true,
        verification_source: 'OEM_VERIFICATION',
```

```
      min_condition: 'Direct authorization letter addressed to tender',
      weight: 15,
      applicable_bidder_type: 'ALL',
    },
    {
      requirement_code: 'MAKE_IN_INDIA',
      requirement_name: 'Make in India (MII) Local Content Declaration',
      is_mandatory: true,
      verification_source: 'DPIIT_MII_DECLARATION',
      min_condition: 'Class-I Local Supplier (>=50% local content)',
      weight: 15,
      applicable_bidder_type: 'ALL',
    },
    {
      requirement_code: 'DEBARMENT',
      requirement_name: 'Non-Blacklisting / Debarment Affidavit',
      is_mandatory: true,
      verification_source: 'CPPP_GEM_DEBARMENT_LIST',
      min_condition: 'No debarment by any Central/State Govt/PSU',
      weight: 10,
      applicable_bidder_type: 'ALL',
    },
  ]),
  [_, S] = useState(false),
  [O, A] = useState('');
useEffect(() => {
  E();
}, []);
const E = async () => {
    try {
      n(true);
      const I = await api.getTenders();
      t(I);
    } catch (I) {
      console.error(I);
    } finally {
      n(false);
    }
  },
  T = async (I) => {
    if ((I.preventDefault(), !f || !p)) {
      A('Title and Department are required.');
      return;
    }
    try {
      (S(true),
        A(''),
        await api.createTender({
          tender_id: c,
          title: f,
          department: p,
          tender_value: y,
          deadline: new Date(g).toISOString(),
          description: m,
          requirements: j,
        }),
        a(false),
        E());
    } catch (R) {
      A(R.message || 'Failed to create tender');
    } finally {
      S(false);
    }
  },
  C = e.filter(
    (I) =>
      I.title.toLowerCase().includes(o.toLowerCase()) ||
      I.tender_id.toLowerCase().includes(o.toLowerCase()) ||
      I.department.toLowerCase().includes(o.toLowerCase()),
  );
return (
  <div className="space-y-6">
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 bg-white p-6 rounded-2xl b
  order border-slate-200/90 shadow-card-soft">
      <div>
        <div className="flex items-center gap-2">
          <h1 className="text-xl font-extrabold text-slate-900 tracking-tight">
            {'Tenders & Evaluation Criteria'}
          </h1>
```

```
        <span className="bg-pine-100 text-pine-800 text-[10px] font-bold px-2.5 py-0.5 rounded-full font-mon
o">
          GFR Rule 149
        </span>
      </div>
      <p className="text-xs text-slate-500 mt-0.5">
        Configure statutory eligibility conditions, scoring weights, and mandatory compliance
        rules.
      </p>
    </div>
    <div className="flex items-center gap-3">
      <button
        onClick={() => a(true)}
        className="px-4 py-2.5 bg-gradient-to-r from-pine-800 to-teal-800 hover:from-pine-900 hover:to-teal-
900 text-white text-xs font-bold rounded-xl shadow-glow-teal transition-all flex items-center gap-2 group"
      >
        <PlusIcon className="w-4 h-4 text-champagne-300" />
        <span>Publish New Tender</span>
      </button>
    </div>
  </div>
  <div className="bg-white p-4 rounded-2xl border border-slate-200/90 shadow-card-soft">
    <div className="relative">
      <SearchIcon className="w-4 h-4 text-slate-400 absolute left-3.5 top-3" />
      <input
        type="text"
        placeholder="Search tenders by Title, Tender ID (e.g. GEM/2026/B/...), Department..."
        value={o}
        onChange={(I) => l(I.target.value)}
        className="w-full pl-10 pr-4 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 focus:bg-wh
ite focus:ring-2 focus:ring-teal-600 focus:outline-none transition-colors"
      />
    </div>
  </div>
  <div className="grid grid-cols-1 md:grid-cols-2 gap-5">
    {r ? (
      <div className="col-span-2 text-center py-12 text-slate-500 text-xs">
        Loading tenders registry...
      </div>
    ) : C.length === 0 ? (
      <div className="col-span-2 text-center py-12 text-slate-500 text-xs bg-white rounded-2xl border border
-dashed border-slate-300 p-8">
        No matching tenders found. Click "Publish New Tender" to create one.
      </div>
    ) : (
      C.map((I) => {
        var R, P, $;
        return (
          <div
            key={I.id}
            className="bg-white rounded-2xl border border-slate-200/90 p-5 shadow-card-soft hover:shadow-car
d-hover transition-all flex flex-col justify-between group"
          >
            <div>
              <div className="flex justify-between items-start mb-2.5">
                <span className="font-mono text-xs font-bold text-slate-900 bg-slate-100 px-2.5 py-1 rounded
-lg border border-slate-200">
                  {I.tender_id}
                </span>
                <span className="text-[10px] font-bold uppercase tracking-wider bg-pine-100 text-pine-800 px
-2.5 py-0.5 rounded-full">
                  {I.status}
                </span>
              </div>
              <h3 className="text-sm font-bold text-slate-900 leading-snug group-hover:text-pine-800 transit
ion-colors">
                {I.title}
              </h3>
              <p className="text-xs text-slate-500 mt-1 flex items-center gap-1.5">
                <Building2Icon className="w-3.5 h-3.5 text-slate-400 flex-shrink-0" />
                <span className="truncate">{I.department}</span>
              </p>
              <div className="grid grid-cols-2 gap-3 mt-4 pt-3 border-t border-slate-100 text-xs">
                <div className="bg-slate-50 p-2.5 rounded-xl border border-slate-100">
                  <span className="text-[10px] text-slate-400 block font-medium">
                    Tender Value:
                  </span>
                  <p className="font-bold font-mono text-slate-900 text-sm">
                    ₹{I.tender_value.toLocaleString('en-IN')}
```

```
                  </p>
                </div>
                <div className="bg-slate-50 p-2.5 rounded-xl border border-slate-100">
                  <span className="text-[10px] text-slate-400 block font-medium">
                    Evaluation Deadline:
                  </span>
                  <p className="font-semibold text-slate-800 text-xs mt-0.5 flex items-center gap-1">
                    <ClockIcon className="w-3.5 h-3.5 text-champagne-600" />
                    {new Date(I.deadline).toLocaleDateString()}
                  </p>
                </div>
              </div>
              <div className="mt-4">
                <span className="text-[10px] font-bold text-slate-400 uppercase tracking-wider block mb-1.5"
>
                  Configured Criteria (
                  {((R = I.requirements) == null ? undefined : R.length) || 0})
                </span>
                <div className="flex flex-wrap gap-1.5">
                  {(P = I.requirements) == null
                    ? undefined
                    : P.slice(0, 5).map((D) => (
                        <span
                          key={D.id || D.requirement_code}
                          className="text-[10px] font-mono bg-slate-100 text-slate-700 px-2 py-0.5 rounded-m
d font-semibold border border-slate-200"
                        >
                          {D.requirement_code}
                        </span>
                      ))}
                  {((($ = I.requirements) == null ? undefined : $.length) || 0) > 5 && (
                    <span className="text-[10px] font-mono text-pine-800 bg-pine-50 px-2 py-0.5 rounded-md f
ont-bold border border-pine-200">
                      +{I.requirements.length - 5}
                      {' more'}
                    </span>
                  )}
                </div>
              </div>
            </div>
            <div className="mt-5 pt-3.5 border-t border-slate-100 flex justify-between items-center text-xs"
>
              <span className="text-slate-500 font-mono text-[11px]">
                <strong>{I.bids_count || 0}</strong>
                {' Submissions'}
              </span>
              <Link
                to={`/bids?tender_id=${I.id}`}
                className="inline-flex items-center gap-1.5 px-3 py-1.5 bg-pine-50 hover:bg-pine-800 text-pi
ne-900 hover:text-white font-bold rounded-xl transition-all border border-pine-200 text-[11px]"
              >
                <span>Review Submissions</span>
                <ChevronRightIcon className="w-3.5 h-3.5" />
              </Link>
            </div>
          </div>
        );
      })
    )}
  </div>
  {i && (
    <div className="fixed inset-0 z-50 bg-slate-950/75 backdrop-blur-sm flex items-center justify-center p-4
">
      <div className="bg-white w-full max-w-4xl max-h-[90vh] rounded-3xl shadow-2xl border border-slate-300
flex flex-col overflow-hidden animate-in fade-in zoom-in-95 duration-150">
        <div className="px-6 py-4 bg-gradient-to-r from-[#060D17] to-[#0D3830] text-white flex justify-betwe
en items-center border-b border-slate-800">
          <div className="flex items-center gap-3">
            <div className="w-9 h-9 rounded-xl bg-pine-800/80 border border-pine-600/50 flex items-center ju
stify-center">
              <FileTextIcon className="w-5 h-5 text-champagne-400" />
            </div>
            <div>
              <h2 className="text-base font-bold">
                {'Publish Tender & Define Criteria Matrix'}
              </h2>
              <p className="text-xs text-slate-300">
                Set mandatory eligibility criteria and scoring weights under GFR 2017
              </p>
```

```
            </div>
          </div>
          <button
            onClick={() => a(false)}
            className="p-1.5 rounded-lg text-slate-400 hover:text-white hover:bg-white/10 transition-colors"
          >
            <XIcon className="w-5 h-5" />
          </button>
        </div>
        <form onSubmit={T} className="flex-1 p-6 overflow-y-auto space-y-5 custom-scrollbar">
          {O && (
            <div className="p-3 bg-rose-50 text-rose-800 rounded-xl text-xs font-semibold border border-rose
-200">
              {O}
            </div>
          )}
          <div className="grid grid-cols-1 sm:grid-cols-2 gap-4 text-xs">
            <div>
              <label className="block font-bold text-slate-700 mb-1">
                Tender ID (GeM Code)
              </label>
              <input
                type="text"
                required
                value={c}
                onChange={(I) => u(I.target.value)}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 font-mono text-xs foc
us:ring-2 focus:ring-teal-600 focus:outline-none"
                />
            </div>
            <div>
              <label className="block font-bold text-slate-700 mb-1">
                Procuring Department / Ministry
              </label>
              <input
                type="text"
                required
                placeholder={'e.g. Department of Electronics & IT'}
                value={p}
                onChange={(I) => h(I.target.value)}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 text-xs focus:ring-2
focus:ring-teal-600 focus:outline-none"
                />
            </div>
            <div className="sm:col-span-2">
              <label className="block font-bold text-slate-700 mb-1">
                Tender Title / Scope of Work
              </label>
              <input
                type="text"
                required
                placeholder={
                  'e.g. Supply, Installation & Maintenance of Enterprise Cloud Server Hardware'
                }
                value={f}
                onChange={(I) => d(I.target.value)}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 text-xs focus:ring-2
focus:ring-teal-600 focus:outline-none"
                />
            </div>
            <div>
              <label className="block font-bold text-slate-700 mb-1">
                Estimated Tender Value (INR ₹)
              </label>
              <input
                type="number"
                required
                value={y}
                onChange={(I) => x(Number(I.target.value))}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 font-mono text-xs foc
us:ring-2 focus:ring-teal-600 focus:outline-none"
                />
            </div>
            <div>
              <label className="block font-bold text-slate-700 mb-1">
                {'Submission & Evaluation Deadline'}
              </label>
              <input
                type="date"
```

```
                  required
                  value={g}
                  onChange={(I) => v(I.target.value)}
                  className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 text-xs focus:ring-2
    focus:ring-teal-600 focus:outline-none"
                 />
               </div>
             </div>
             <div className="pt-4 border-t border-slate-200">
               <RequirementsEditor requirements={j} onChange={w} />
             </div>
             <div className="flex justify-end items-center gap-3 pt-4 border-t border-slate-200">
               <button
                 type="button"
                 onClick={() => a(false)}
                 className="px-4 py-2.5 text-xs font-semibold text-slate-600 hover:bg-slate-100 rounded-xl tran
    sition-colors"
               >
                 Cancel
               </button>
               <button
                 type="submit"
                 disabled={_}
                 className="px-6 py-2.5 text-xs font-bold text-white bg-gradient-to-r from-pine-800 to-teal-800
     hover:from-pine-900 hover:to-teal-900 rounded-xl shadow-glow-teal transition-all"
               >
                 {_ ? 'Publishing Tender...' : 'Publish Tender & Criteria'}
               </button>
             </div>
           </form>
         </div>
       </div>
     )}
   </div>
  );
};


//// FILE: src/routes/ProtectedRoute.jsx  (7 lines)

import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export const ProtectedRoute = ({ children: e }) => {
  const { isAuthenticated: t } = useAuth();
  return t ? <>{e}</> : <Navigate to="/login" replace />;
};


//// FILE: start-all.bat  (4 lines)

@echo off
rem Opens the backend and the frontend, each in its own window
start "GeM BidVerify - backend" cmd /k "%~dp0start-backend.bat"
start "GeM BidVerify - frontend" cmd /k "%~dp0start-frontend.bat"


//// FILE: start-backend.bat  (13 lines)

@echo off
rem Starts the FastAPI backend on http://127.0.0.1:8000  (API docs: http://127.0.0.1:8000/docs)
rem Optional e-mail settings live in secrets.bat (copy secrets.bat.example to secrets.bat and edit it).
cd /d "%~dp0backend"
if exist "%~dp0secrets.bat" call "%~dp0secrets.bat"
if not exist .venv (
  echo Creating Python virtual environment...
  py -3 -m venv .venv 2>nul || python -m venv .venv
)
call .venv\Scripts\activate.bat
python -m pip install --quiet -r requirements.txt
python -m uvicorn app.main:app --reload --port 8000
pause


//// FILE: start-frontend.bat  (6 lines)

@echo off
rem Starts the React app on http://localhost:5173
cd /d "%~dp0"
```

```
if not exist node_modules call npm install
call npm run dev
pause


//// FILE: tailwind.config.js  (47 lines)

/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,jsx}'],
  theme: {
    extend: {
      fontFamily: {
        sans: ['Plus Jakarta Sans', 'Inter', 'system-ui', '-apple-system', 'sans-serif'],
        mono: ['JetBrains Mono', 'Fira Code', 'monospace'],
      },
      colors: {
        "champagne": {
              "50": "#FFFBEB",
              "100": "#FEF3C7",
              "200": "#FDE68A",
              "300": "#FCD34D",
              "400": "#F59E0B",
              "500": "#D97706",
              "600": "#C2410C",
              "700": "#9A3412",
              "800": "#78350F",
              "900": "#451A03"
        },
        "pine": {
              "400": "#2DD4BF",
              "500": "#14B8A6",
              "600": "#0F766E",
              "700": "#134E43",
              "800": "#0D3830",
              "900": "#08231E",
              "950": "#041411"
        },
        "obsidian": {
              "850": "#0E1C2F",
              "900": "#0B1524",
              "950": "#060D17"
        }
  },
      boxShadow: {
        'card-soft': '0 2px 10px -2px rgba(0, 0, 0, .04), 0 1px 3px -1px rgba(0, 0, 0, .02)',
        'card-hover': '0 12px 24px -6px rgba(11, 21, 36, .08), 0 4px 8px -2px rgba(11, 21, 36, .04)',
        'glow-teal': '0 0 20px -3px rgba(20, 184, 166, .25)',
        'glow-champagne': '0 0 20px -3px rgba(245, 158, 11, .25)',
      },
    },
  },
  plugins: [],
};


//// FILE: vite.config.js  (13 lines)

import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import { viteSingleFile } from 'vite-plugin-singlefile';

// `npm run dev`   -> dev server with hot reload; requests to /api are forwarded to the backend on port 8000
// `npm run build` -> dist/index.html as ONE self-contained file
const backend = 'http://127.0.0.1:8000';

export default defineConfig({
  plugins: [react(), viteSingleFile()],
  server: { port: 5173, proxy: { '/api': { target: backend, changeOrigin: true } } },
  preview: { proxy: { '/api': { target: backend, changeOrigin: true } } },
});
```